

Documento de Especificación de Arquitectura de Software: Modelo 4 + 1

Misión Oasis Lunar (Simulador Artemis II)

Versión 1.0 aprobada

Preparado por

CDR: Laura Priscila Guerrero Echavarría

REQ: Yanantonys Germosen López

ARCH: Patrick Schriels Pérez

FDO: Rommel Arturo Santana Arias

CAPCOM: Martín Isaac Jiménez Gabot

Universidad Nacional Pedro Henríquez Ureña (UNPHU)

29/6/2026

Tabla de Contenido

1. Introducción.....	3
1.1 Alcance del producto	3
1.2 Definiciones, acrónimos y abreviaturas	3
1.3 Referencias.....	4
2. Representación arquitectónica	5
2.1 El Modelo 4+1.....	5
2.2 Representación gráfica del Modelo 4+1 aplicada al proyecto	5
2.3 Descripción general de la arquitectura de Misión Oasis Lunar.....	6
2.4 Matriz de trazabilidad de requisitos	7
2.5 Decisiones arquitectónicas (ADR)	8
3. Objetivos y restricciones arquitectónicas.....	10
4. Escenarios (casos de uso)	11
4.1 Diagramas de Casos de Uso.....	11
4.2 Diagramas de Secuencia.....	14
5. Vista lógica.....	18
6. Vista de proceso.....	21
7. Vista de desarrollo (implementación).....	24
8. Vista física	26
9. Conclusión	28

1. Introducción

1.1 Alcance del producto

El presente documento describe la arquitectura de software del sistema Misión Oasis Lunar, un simulador aeroespacial educativo inspirado en la misión Artemis II de la NASA. La arquitectura propuesta define la organización del sistema, la interacción entre componentes, las tecnologías utilizadas y la distribución de responsabilidades entre los diferentes módulos.

El objetivo principal de esta especificación es proporcionar una visión estructurada del diseño del software utilizando el modelo arquitectónico 4+1, permitiendo analizar el sistema desde diferentes perspectivas: lógica, proceso, desarrollo, física y escenarios de uso.

El sistema busca simular una misión translunar mediante el cálculo de trayectorias orbitales, visualización gráfica, monitoreo de telemetría y generación de alertas operativas. Para lograrlo, integra tecnologías como Orekit, JavaFX y la plataforma Java Virtual Machine (JVM).

Con respecto al Entregable #1 (Documento de Especificación de Requisitos de Software, SRS), esta versión traduce los requisitos funcionales allí definidos —control de simulación, cálculo orbital, monitoreo de telemetría y generación de alertas— en una estructura arquitectónica concreta, estableciendo los componentes de software responsables de satisfacer cada requisito y sus relaciones.

1.2 Definiciones, acrónimos y abreviaturas

Término	Definición
4+1	Modelo de vistas arquitectónicas propuesto por Philippe Kruchten, compuesto por las vistas lógica, de procesos, de desarrollo, física y de escenarios.
UML	Unified Modeling Language — lenguaje de modelado utilizado para representar los diagramas de esta especificación.
Orekit	Biblioteca de código abierto en Java para cálculo y propagación de órbitas espaciales.
JavaFX	Framework de Java utilizado para el desarrollo de la interfaz gráfica de escritorio del simulador.
JVM	Java Virtual Machine — entorno de ejecución sobre el cual corre la aplicación.
SRS	Software Requirements Specification — documento de especificación de requisitos de software (Entregable #1).
CDR	Commander — rol de dirección general del proyecto dentro del equipo.
REQ	Requirements — rol responsable de la gestión de requisitos del proyecto.
ARCH	Architect — rol responsable del diseño y especificación de la arquitectura de software.
FDO	Flight Dynamics Officer — rol responsable de la lógica y cálculos de dinámica orbital.

CAPCOM	Capsule Communicator — rol responsable de la comunicación, documentación y presentación del proyecto.
Telemetría	Conjunto de datos operativos (velocidad, altitud, tiempo, estado) transmitidos desde el motor orbital hacia la interfaz de usuario.

1.3 Referencias

- Guerrero Echavarría, L., Germosen López, Y., Schriels Pérez, P., Santana Arias, R. y Jiménez Gabot, M. (2026). Documento de Especificación de Requisitos de Software (SRS) — Misión Oasis Lunar, Entregable #1. UNPHU.
- Kruchten, P. (1995). Architectural Blueprints — The "4+1" View Model of Software Architecture. IEEE Software, 12(6), 42–50.
- Orekit — An accurate and efficient core layer for space flight dynamics applications. Documentación oficial: <https://www.orekit.org>
- Oracle / OpenJFX. JavaFX Documentation. <https://openjfx.io>
- NASA. Artemis II Mission Overview. <https://www.nasa.gov/artemis-ii>
- Object Management Group (OMG). Unified Modeling Language (UML) Specification. <https://www.omg.org/spec/UML>

2. Representación arquitectónica

2.1 El Modelo 4+1

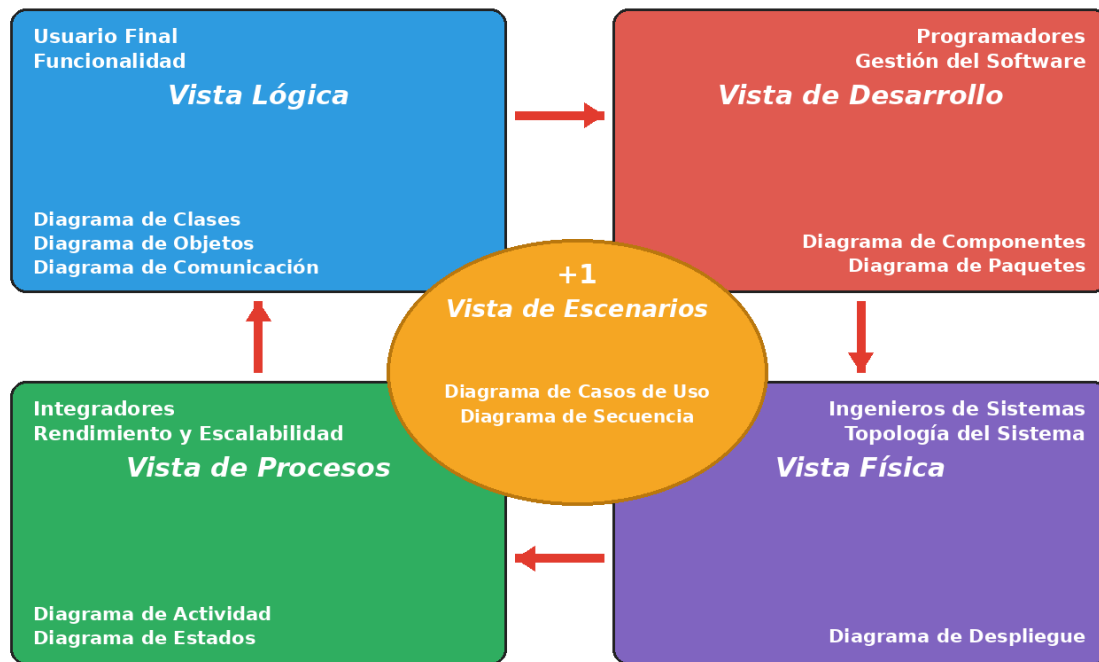
El Modelo 4+1, propuesto por Philippe Kruchten, es un enfoque para describir la arquitectura de sistemas de software complejos mediante múltiples vistas concurrentes, cada una enfocada en un conjunto particular de preocupaciones (stakeholders). En lugar de intentar capturar toda la arquitectura en un único diagrama, el modelo separa la descripción del sistema en cuatro vistas principales, complementadas por una quinta vista de escenarios que las integra y valida:

- Vista lógica: describe las abstracciones clave del sistema —clases, objetos y sus relaciones— y satisface los requisitos funcionales.
- Vista de proceso: describe los aspectos de concurrencia, sincronización y comportamiento en tiempo de ejecución del sistema.
- Vista de desarrollo: describe la organización estática del software en el entorno de desarrollo, incluyendo módulos, paquetes y librerías.
- Vista física: describe el mapeo del software sobre el hardware y la infraestructura de despliegue.
- Vista de escenarios (+1): un conjunto de casos de uso o escenarios que ilustran y validan la coherencia entre las cuatro vistas anteriores.

Esta separación de perspectivas permite que distintos interesados —usuarios finales, desarrolladores, integradores de sistemas e ingenieros— comprendan la arquitectura desde el ángulo que les resulta más relevante, sin perder la coherencia global del diseño.

2.2 Representación gráfica del Modelo 4+1 aplicada al proyecto

La siguiente figura resume cómo se aplica el modelo 4+1 al sistema Misión Oasis Lunar: cada vista se asocia con su interesado principal y con los diagramas UML desarrollados en este documento para representarla, mientras que la vista de escenarios (+1), en el centro, integra y valida a las cuatro.



Representación gráfica del Modelo 4+1 — Misión Oasis Lunar.

2.3 Descripción general de la arquitectura de Misión Oasis Lunar

Aplicando el modelo 4+1 al simulador Misión Oasis Lunar, el sistema está compuesto por los siguientes componentes principales:

Motor Orbital

Responsable de realizar los cálculos físicos relacionados con la trayectoria de la nave utilizando Orekit.

Funciones principales:

- Propagación orbital.
- Cálculo de posición y velocidad.
- Actualización del estado de la nave.
- Manejo de referencias temporales.

Controlador de Simulación

Administra el ciclo de vida de la misión.

Funciones principales:

- Iniciar simulación.
- Pausar ejecución.
- Reiniciar escenarios.

- Controlar tiempo de misión.

Sistema de Telemetría

Procesa los datos generados por el motor orbital y los presenta al usuario.

Información mostrada:

- Velocidad orbital.
- Altitud lunar.
- Tiempo transcurrido.
- Estado operacional.

Sistema de Alertas

Supervisa los parámetros de la misión y genera avisos cuando se detectan condiciones anormales.

Interfaz Gráfica

Desarrollada con JavaFX. Permite:

- Visualización orbital.
- Control del sistema.
- Consulta de telemetría.
- Visualización de alertas.

2.4 Matriz de trazabilidad de requisitos

La siguiente matriz confirma que cada requisito funcional principal está cubierto por al menos un caso de uso y un artefacto UML en cada una de las cinco vistas del modelo 4+1.

Requisito funcional	Caso de uso	Escenarios (+1)	Lógica	Proceso	Desarrollo	Física
Iniciar/gestionar el ciclo de vida de la simulación	UC1	Fig. 1, Fig. 8	Fig. 15 (ControlSimulacion)	Fig. 17, Fig. 21	Fig. 22 (SimulationController)	Fig. 23
Calcular y propagar la trayectoria orbital	UC4	Fig. 4, Fig. 14	Fig. 15 (MotorOrbital)	Fig. 18	Fig. 22 (OrbitalEngine)	Fig. 23
Consultar telemetría de la misión	UC2	Fig. 2, Fig. 13	Fig. 15 (Telemetry)	Fig. 17	Fig. 22 (TelemetryManager)	Fig. 23
Generar y visualizar alertas operativas	UC3	Fig. 3, Fig. 10	Fig. 15 (GestorAlertas)	Fig. 19	Fig. 22 (AlertManager)	Fig. 23
Configurar el escenario de misión	UC5, UC7	Fig. 5, Fig. 7	Fig. 15 (SimuladorMision)	Fig. 17	Fig. 22 (SimulationController)	Fig. 23

2.5 Decisiones arquitectónicas (ADR)

Se documentan a continuación las decisiones arquitectónicas más relevantes del proyecto, siguiendo un formato abreviado de Architecture Decision Record (ADR): contexto, alternativas consideradas, decisión tomada y consecuencias.

ADR-01 — Uso de Orekit como motor de cálculo orbital

Contexto: El sistema requiere propagación orbital precisa (posición, velocidad, referencias temporales) sin implementar mecánica orbital desde cero.

Alternativas consideradas: Implementar un motor propio de mecánica orbital; usar otra librería (GMAT, Skyfield en Python).

Justificación: Orekit es una librería Java madura, de código abierto, con soporte activo del OMG/CS y usada en misiones reales; se integra nativamente con el resto del stack Java del proyecto.

Consecuencias: El sistema queda acoplado a la API y al modelo de datos de Orekit; se representa como paquete «library» (no como clase) en el diagrama de clases y de componentes.

ADR-02 — JavaFX para la interfaz gráfica de escritorio

Contexto: Se necesita una interfaz gráfica de escritorio capaz de mostrar visualización orbital y paneles de control/telemetría en tiempo real.

Alternativas consideradas: Swing (más antiguo, menos capacidades gráficas); interfaz web (requeriría servidor y complejidad adicional no justificada para v1).

Justificación: JavaFX ofrece gráficos vectoriales y animación fluida, se mantiene activamente vía OpenJFX y es la opción recomendada para nuevas aplicaciones de escritorio en Java.

Consecuencias: Todos los componentes de la capa de presentación (MainView, ControlsPanel) dependen de JavaFX; una futura migración a interfaz web requeriría rediseñar esta capa.

ADR-03 — Arquitectura de escritorio monolítica en una sola máquina para la versión 1.0

Contexto: El alcance inicial del proyecto es un simulador educativo de un único usuario, sin requisito de colaboración remota.

Alternativas consideradas: Arquitectura cliente-servidor con backend remoto; arquitectura distribuida para simular múltiples vehículos.

Justificación: Minimiza la complejidad de despliegue y el tiempo de desarrollo para el alcance actual, evitando incurrir en costos de infraestructura de red no requeridos por los objetivos del SRS.

Consecuencias: La vista física no requiere nodos de red ni protocolos de comunicación remota; una evolución futura hacia colaboración multiusuario exigiría revisar esta decisión.

ADR-04 — Separación de responsabilidades en paquetes por capa

Contexto: El sistema combina lógica de dominio (orbital), aplicación (control, telemetría, alertas) y presentación (UI), que deben poder evolucionar de forma independiente.

Alternativas consideradas: Organizar el código en un único paquete plano; separar solo por tipo de clase (modelos, controladores) sin capas explícitas.

Justificación: La separación en capas con interfaces provistas/requeridas (Fig. 22) reduce el acoplamiento, facilita las pruebas unitarias por capa y evita dependencias circulares.

Consecuencias: Incrementa el número de paquetes e interfaces a mantener, pero mejora la mantenibilidad y la trazabilidad de requisitos por componente.

ADR-05 — Ejecución multihilo para separar cálculo orbital de la interfaz

Contexto: El hilo de UI de JavaFX no debe bloquearse mientras Orekit calcula la propagación orbital en cada ciclo de actualización.

Alternativas consideradas: Ejecutar todo en el hilo principal de la aplicación (bloqueante); usar un hilo por cada subsistema (telemetría, alertas, orbital) de forma independiente.

Justificación: Un hilo de simulación dedicado que notifica a la UI de forma asíncrona (ver mensajes asíncronos en Fig. 8 y Fig. 9) mantiene la interfaz responsiva sin sobre-complicar la concurrencia con múltiples hilos.

Consecuencias: Requiere sincronización cuidadosa al actualizar EstadoOrbital desde el hilo de simulación y leerlo desde el hilo de UI.

3. Objetivos y restricciones arquitectónicas

La arquitectura tiene como finalidad garantizar que el simulador sea modular, mantenible, escalable y capaz de incorporar futuras mejoras relacionadas con nuevas misiones espaciales.

Los objetivos principales son:

- Separar las responsabilidades del sistema mediante módulos independientes.
- Permitir la actualización de datos orbitales en tiempo real.
- Facilitar la comunicación entre simulación, interfaz gráfica y monitoreo.
- Garantizar estabilidad mediante ejecución multihilo.
- Permitir futuras extensiones como visualización 3D o simulación de maniobras avanzadas.

La arquitectura sigue principios de programación orientada a objetos, bajo una estructura desacoplada que reduce la dependencia entre componentes.

Como restricciones arquitectónicas se identifican las siguientes:

- El sistema debe ejecutarse sobre la JVM, lo que condiciona el rendimiento a las características del entorno Java.
- El cálculo orbital depende de la biblioteca externa Orekit, por lo que su precisión y comportamiento están limitados por dicha dependencia.
- La primera versión del sistema está restringida a una única máquina de escritorio, sin soporte de red ni distribución.
- La interfaz gráfica debe implementarse con JavaFX, lo que determina las capacidades visuales disponibles para la versión actual.

4. Escenarios (casos de uso)

A continuación se presentan los diagramas de casos de uso que ilustran las principales interacciones del Usuario (Operador de Misión) con el sistema Misión Oasis Lunar, seguidos de los diagramas de secuencia que detallan el comportamiento interno de cada escenario.

4.1 Diagramas de Casos de Uso

Caso de Uso 1 — Gestión de la Simulación

Permite al usuario controlar el ciclo de vida de la simulación: iniciar, pausar, reiniciar y controlar el tiempo de misión.

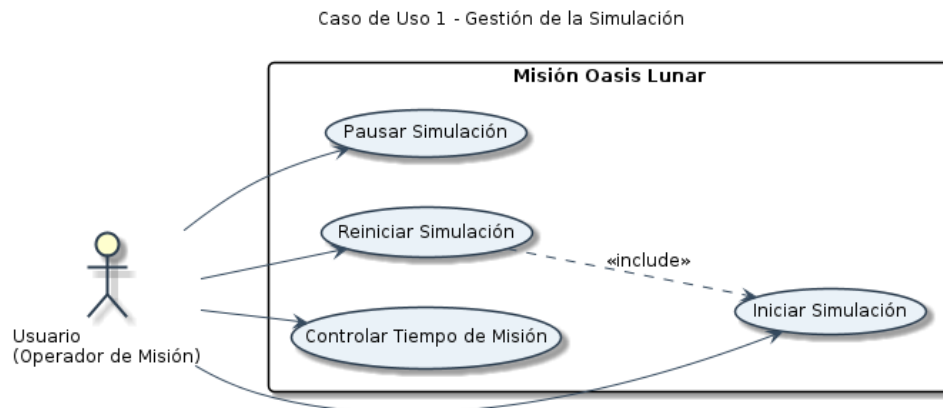


Figura 1. Diagrama de casos de uso — Gestión de la simulación.

Caso de Uso 2 — Consulta de Telemetría

El usuario consulta los datos de telemetría generados a partir del estado orbital: velocidad, altitud, tiempo transcurrido y estado operacional.

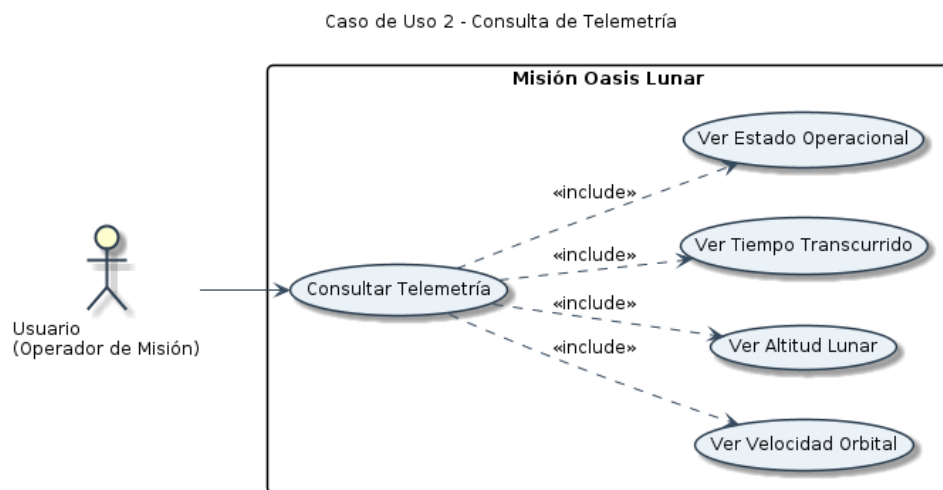


Figura 2. Diagrama de casos de uso — Consulta de telemetría.

Caso de Uso 3 — Alertas Operativas

El Motor Orbital genera alertas automáticas cuando se detectan condiciones anormales; el usuario las visualiza en la interfaz.

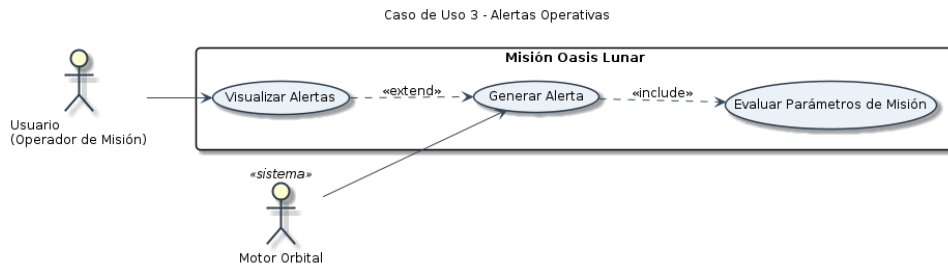


Figura 3. Diagrama de casos de uso — Alertas operativas.

Caso de Uso 4 — Visualización Orbital

El usuario visualiza la trayectoria orbital de la nave y consulta su posición y velocidad en tiempo real.

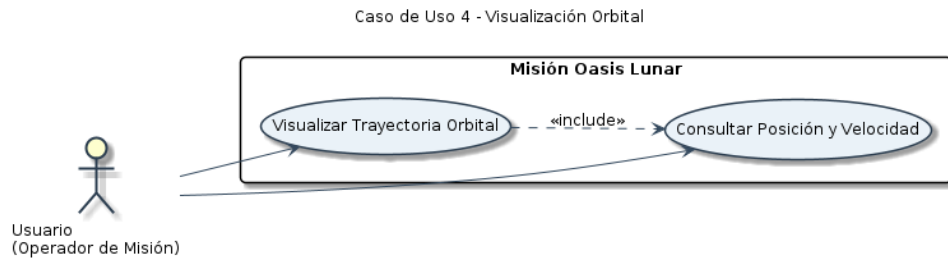


Figura 4. Diagrama de casos de uso — Visualización orbital.

Caso de Uso 5 — Configuración del Escenario

El usuario configura el escenario de misión y carga los parámetros iniciales antes de comenzar la simulación.

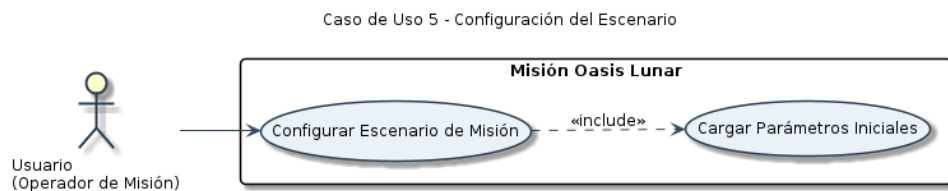


Figura 5. Diagrama de casos de uso — Configuración del escenario.

Caso de Uso 6 — Diagrama General del Sistema

Vista consolidada de todos los casos de uso del sistema y su actor principal.

Caso de Uso 6 - Diagrama General del Sistema

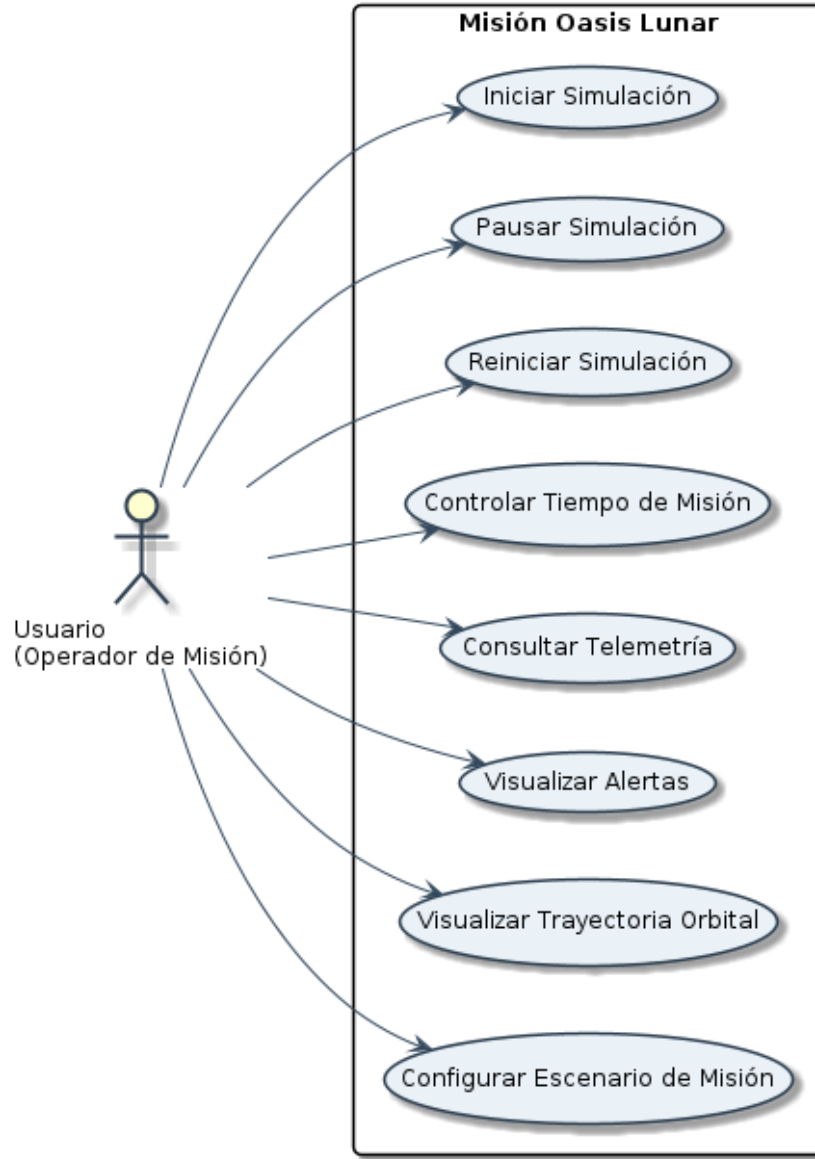


Figura 6. Diagrama de casos de uso general — Misión Oasis Lunar.

Caso de Uso 7 — Generalización de Actores y Casos de Uso

Se identifican dos tipos de operador que especializan al actor genérico "Operador de Misión": el Operador Novato, que trabaja con escenarios predefinidos, y el Operador Experto, que puede personalizar completamente el escenario. De forma análoga, el caso de uso "Configurar Escenario de Misión" se generaliza en dos variantes especializadas.

Caso de Uso 7 - Generalización de Actores y Casos de Uso

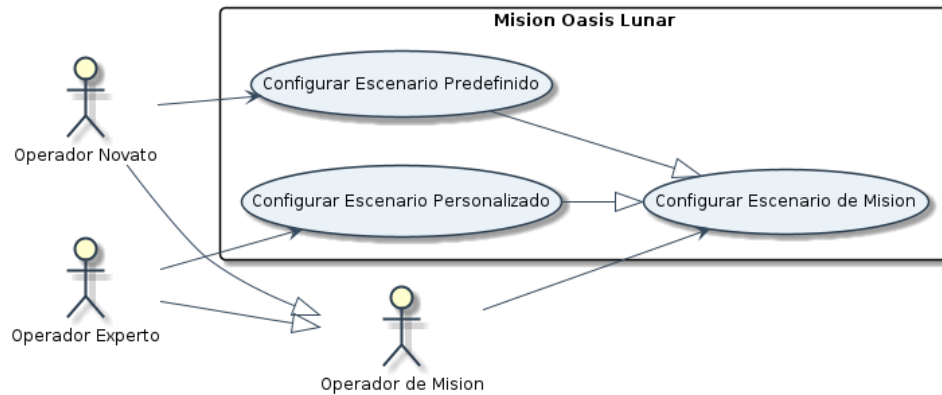


Figura 7. Diagrama de casos de uso — Generalización de actores y casos de uso.

4.2 Diagramas de Secuencia

Los siguientes diagramas de secuencia describen, paso a paso, la interacción entre los objetos del sistema para cada escenario clave, en correspondencia con los procesos descritos en la Vista de Proceso. Se distinguen mensajes síncronos (flecha llena), de retorno (flecha punteada) y asíncronos (flecha abierta, por ejemplo la notificación de telemetría que no bloquea al emisor); se utilizan contenedores loop para flujos repetitivos, opt para lógica condicional simple y alt para bifurcaciones con rutas alternativas.

Secuencia 1 — Inicio de Simulación

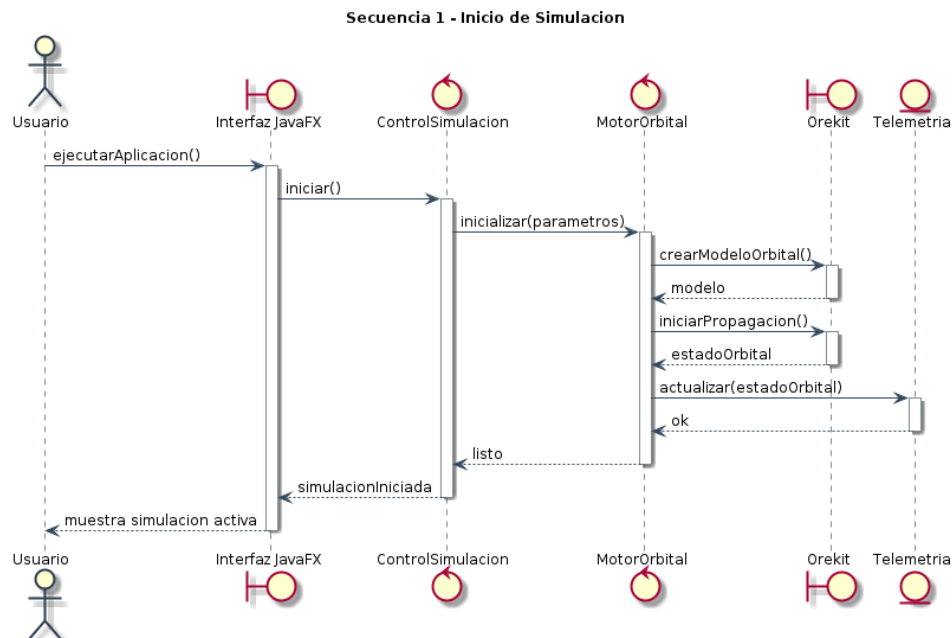


Figura 8. Diagrama de secuencia — Inicio de simulación.

Secuencia 2 — Actualización Orbital

Este escenario se repite continuamente mientras la simulación está en estado EJECUTANDO, por lo que se modela con un contenedor loop.

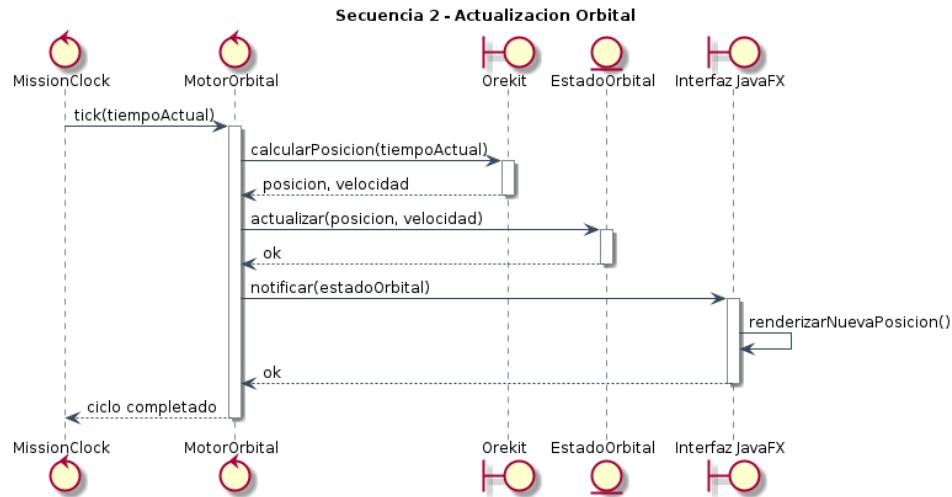


Figura 9. Diagrama de secuencia — Actualización orbital.

Secuencia 3 — Generación de Alertas

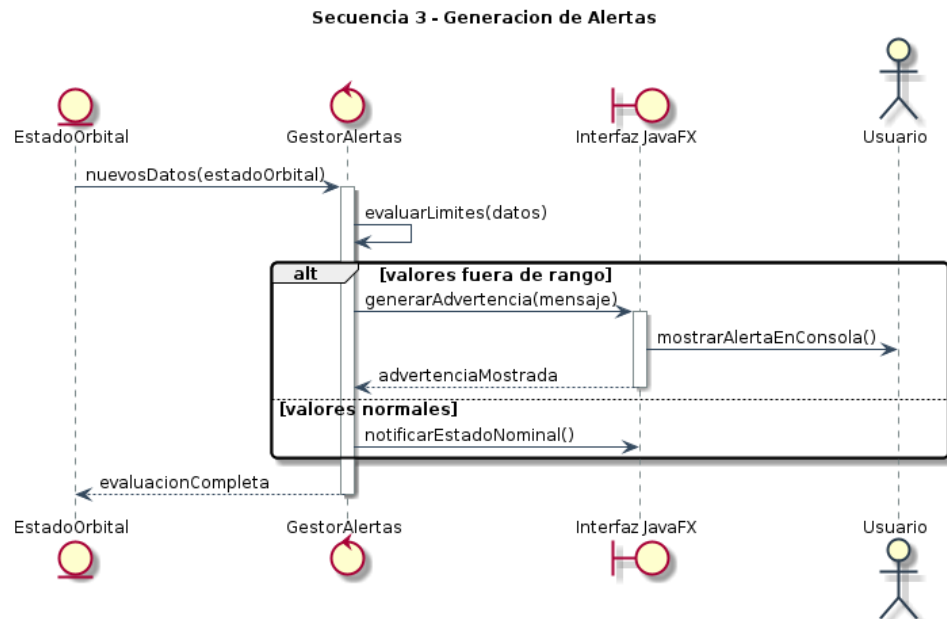


Figura 10. Diagrama de secuencia — Generación de alertas.

Secuencia 4 — Reinicio de la Simulación

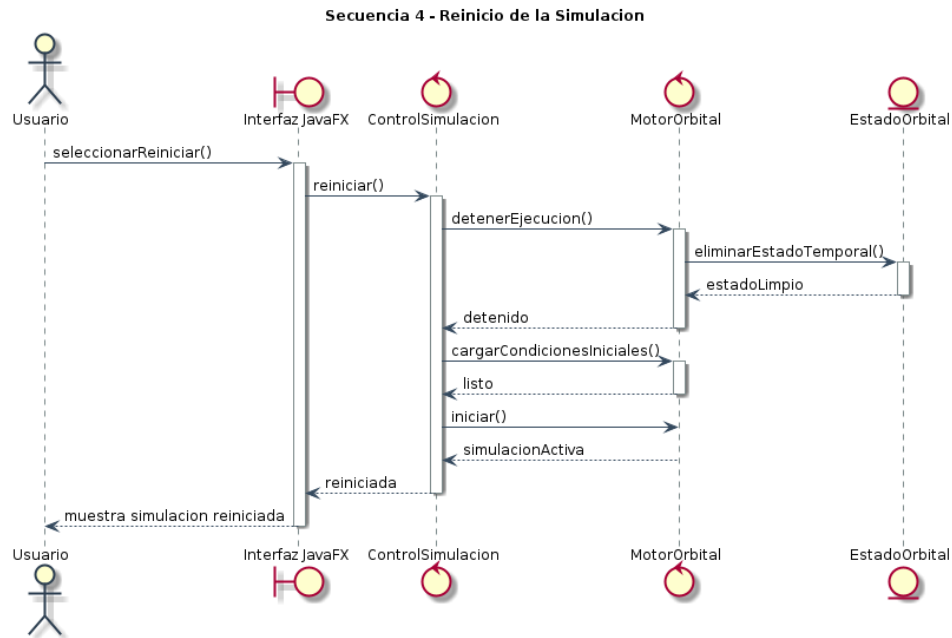


Figura 11. Diagrama de secuencia — Reinicio de la simulación.

Secuencia 5 — Pausar Simulación

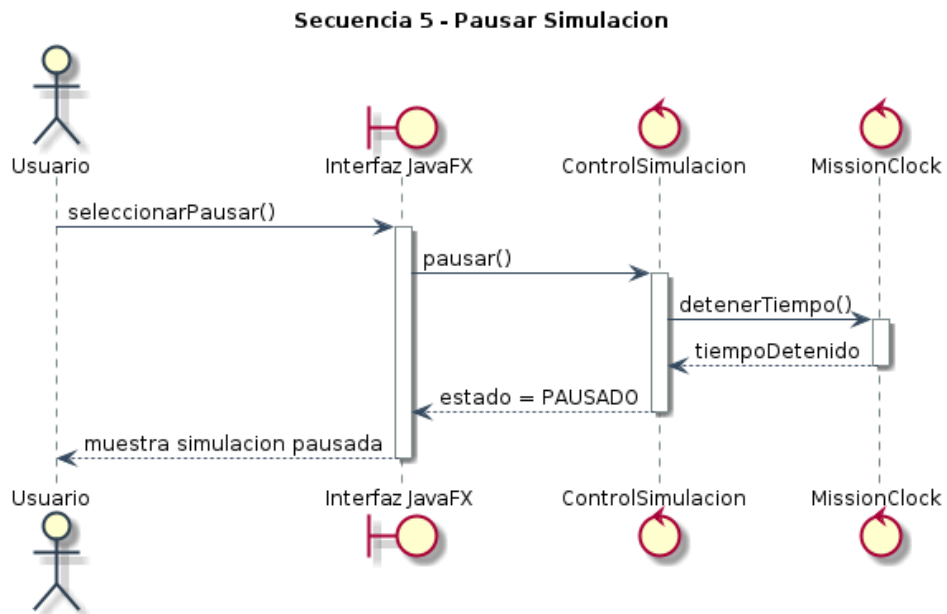


Figura 12. Diagrama de secuencia — Pausar simulación.

Secuencia 6 — Consulta de Telemetría

Incluye un bloque opt que consulta alertas activas únicamente cuando existen, para resaltar los valores correspondientes en el panel de telemetría.

Secuencia 6 - Consulta de Telemetría

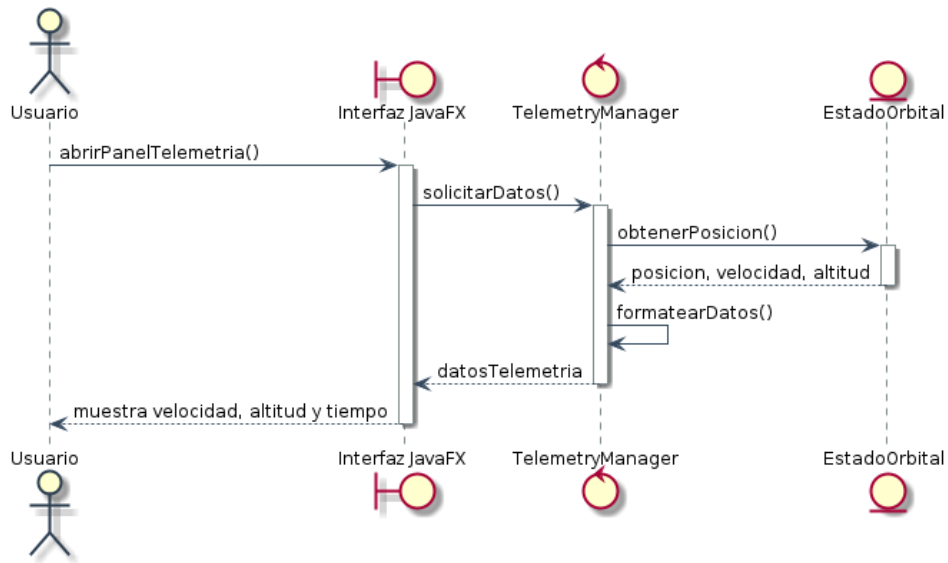


Figura 13. Diagrama de secuencia — Consulta de telemetría.

Secuencia 7 — Visualización de Trayectoria Orbital

Secuencia 7 - Visualización de Trayectoria Orbital

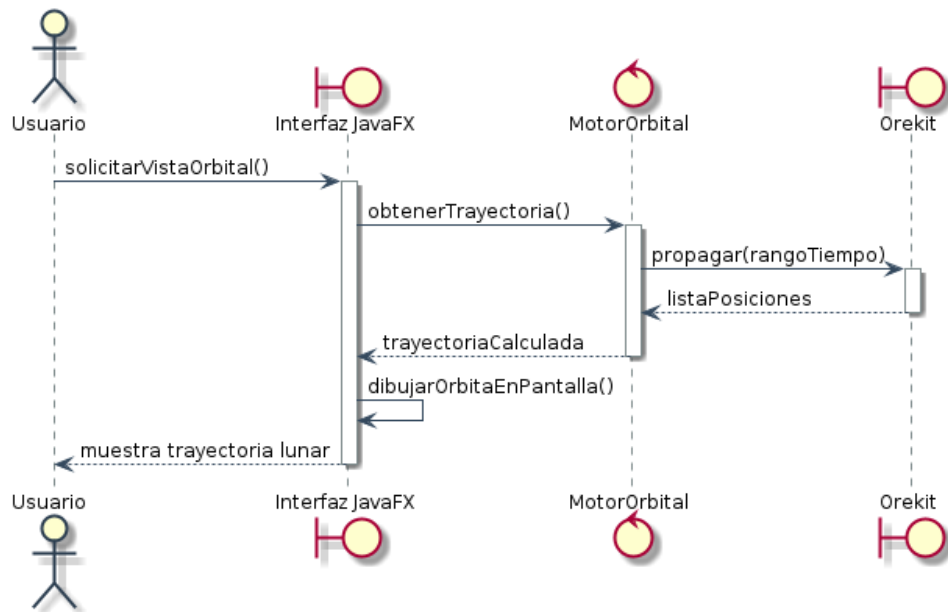


Figura 14. Diagrama de secuencia — Visualización de trayectoria orbital.

5. Vista lógica

La vista lógica representa la estructura funcional del sistema y las relaciones entre sus principales clases y componentes. La arquitectura está organizada utilizando una separación por responsabilidades.

Componentes principales

Clase `SimuladorMision`

Representa el punto principal de ejecución del sistema.

Responsabilidades:

- Inicializar componentes.
- Coordinar la ejecución.
- Gestionar configuración inicial.

Clase `MotorOrbital`

Encargada de comunicarse con Orekit.

Responsabilidades:

- Crear modelos orbitales.
- Ejecutar propagaciones.
- Entregar datos calculados.

Clase `EstadoOrbital`

Representa la información actual de la nave.

Atributos principales:

- Posición.
- Velocidad.
- Tiempo.
- Altitud.

Clase `Telemetry`

Procesa los datos orbitales y los prepara para mostrar al usuario.

Clase `ControlSimulacion`

Administra acciones del usuario.

Métodos principales: `iniciar()`, `pausar()`, `reiniciar()`.

Clase `GestorAlertas`

Evalúa condiciones críticas y genera mensajes.

Relaciones principales

- Usuario → Interfaz JavaFX → ControlSimulacion → MotorOrbital → Orekit
- MotorOrbital → EstadoOrbital → Telemetria
- EstadoOrbital → GestorAlertas

La comunicación entre módulos se realiza mediante objetos de datos compartidos para mantener bajo acoplamiento.

Diagrama de Clases

Nota de notación: Orekit se representa como paquete «library», no como clase, ya que corresponde a un conjunto de módulos y paquetes externos y no a una única clase del sistema.

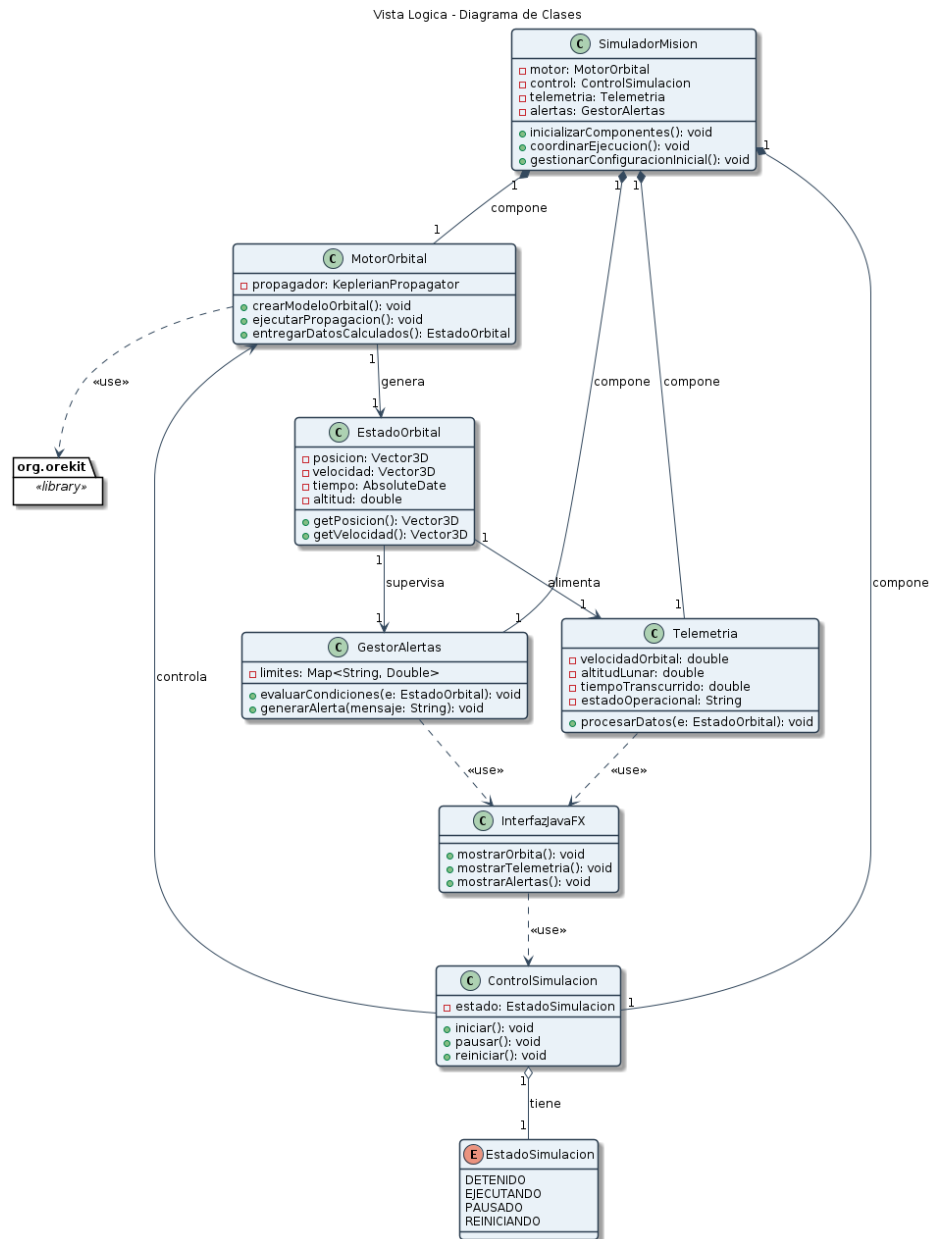


Figura 15. Diagrama de clases — Misión Oasis Lunar.

Diagrama de Objetos

Ejemplo de instancias del sistema en un punto específico de la simulación (T+02:14:36).

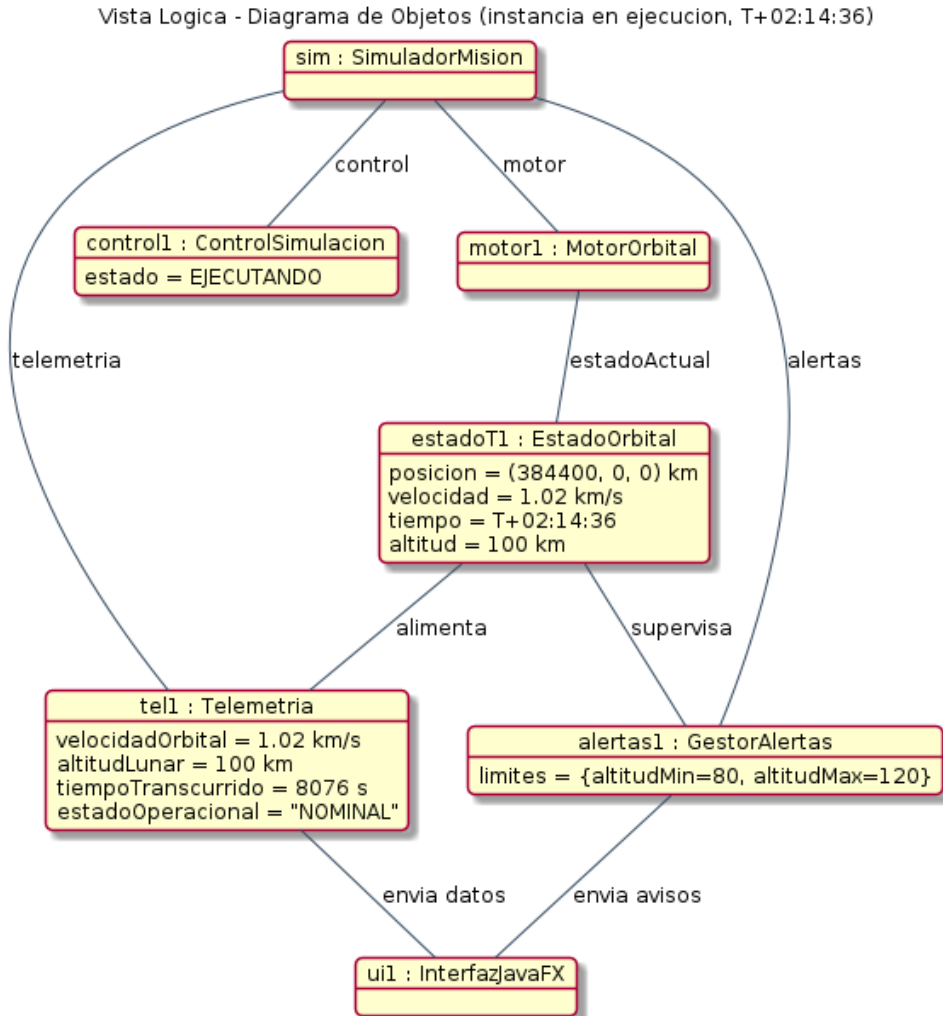


Figura 16. Diagrama de objetos — instantánea de ejecución.

6. Vista de proceso

La vista de proceso describe el comportamiento del sistema durante la ejecución.

Proceso de Inicio de Simulación

- El usuario ejecuta la aplicación.
- La interfaz carga los componentes principales.
- El controlador inicializa el escenario.
- El motor orbital carga parámetros iniciales.
- Orekit inicia la propagación.
- Se generan datos orbitales.
- Telemetría y visualización son actualizadas.

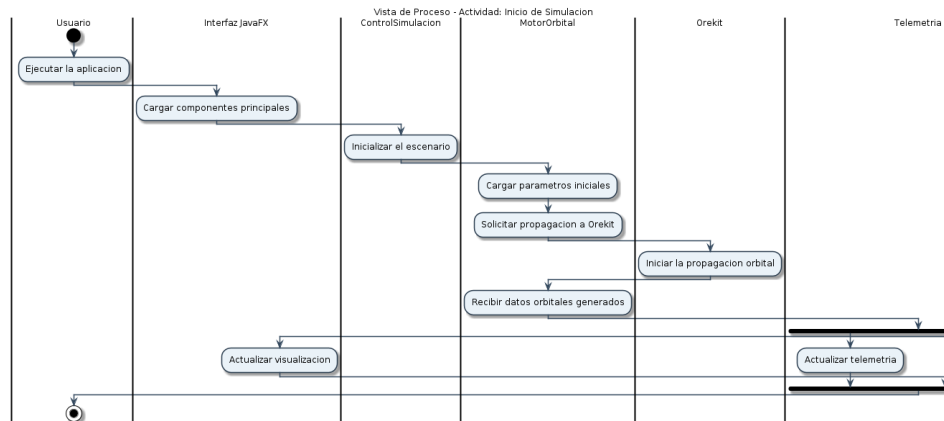


Figura 17. Diagrama de actividad — Inicio de simulación.

Proceso de Actualización Orbital

- El motor orbital recibe el tiempo actual.
- Orekit calcula la nueva posición.
- El estado orbital es actualizado.
- La interfaz gráfica recibe nuevos datos.
- La representación cambia en pantalla.

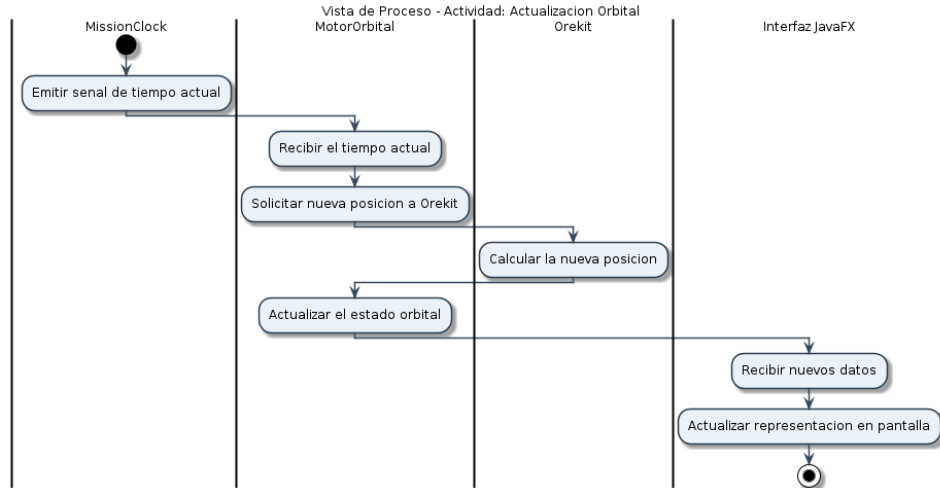


Figura 18. Diagrama de actividad — Actualización orbital.

Proceso de Generación de Alertas

- El sistema recibe nuevos datos orbitales.
- Se comparan valores contra límites establecidos.
- Si existe una desviación: se genera advertencia, se actualiza la consola y se informa al usuario.

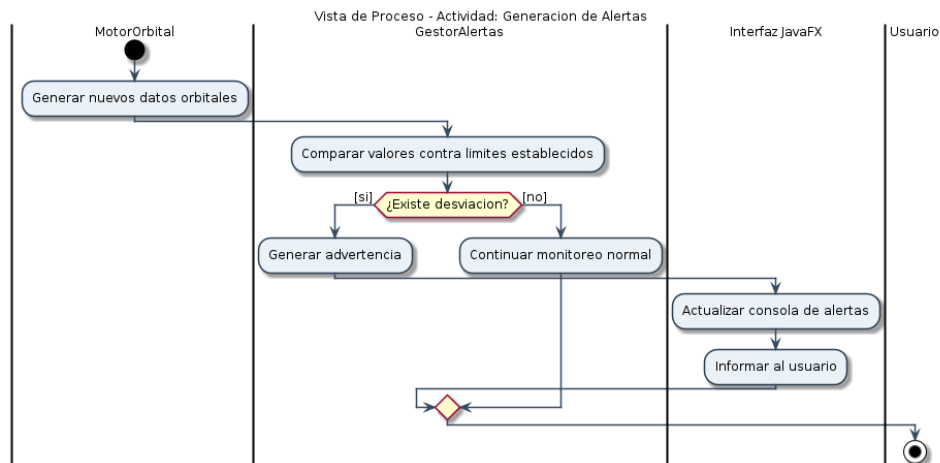


Figura 19. Diagrama de actividad — Generación de alertas.

Proceso de Reinicio

- Usuario selecciona reiniciar.
- El controlador detiene la ejecución actual.
- Se eliminan estados temporales.
- Se cargan condiciones iniciales.
- La simulación comienza nuevamente.

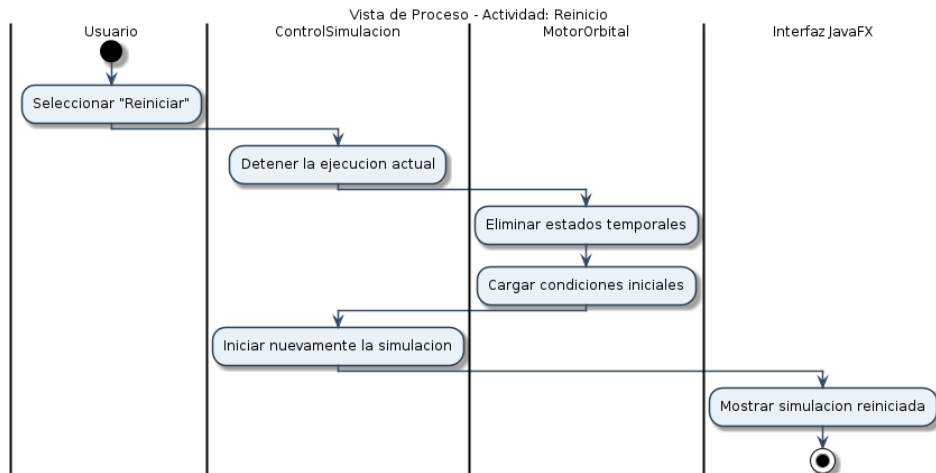


Figura 20. Diagrama de actividad — Reinicio de la simulación.

Diagrama de Estados de la Simulación

Resume las transiciones entre los estados globales del sistema: Detenido, Ejecutando, Pausado y Reiniciando. Al igual que en los diagramas de actividad, los estados se agrupan en regiones según el componente responsable: ControlSimulacion (Detenido, Ejecutando, Pausado) y MotorOrbital (Reiniciando), manteniendo el mismo estilo visual y de agrupación por responsable. La transición iniciar() se dispara desde UC1 (Figura 1) y corresponde a la Secuencia 1 (Figura 8); pausar() y reanudar() corresponden a la Secuencia 5 (Figura 12); reiniciar() corresponde a la Secuencia 4 (Figura 11).

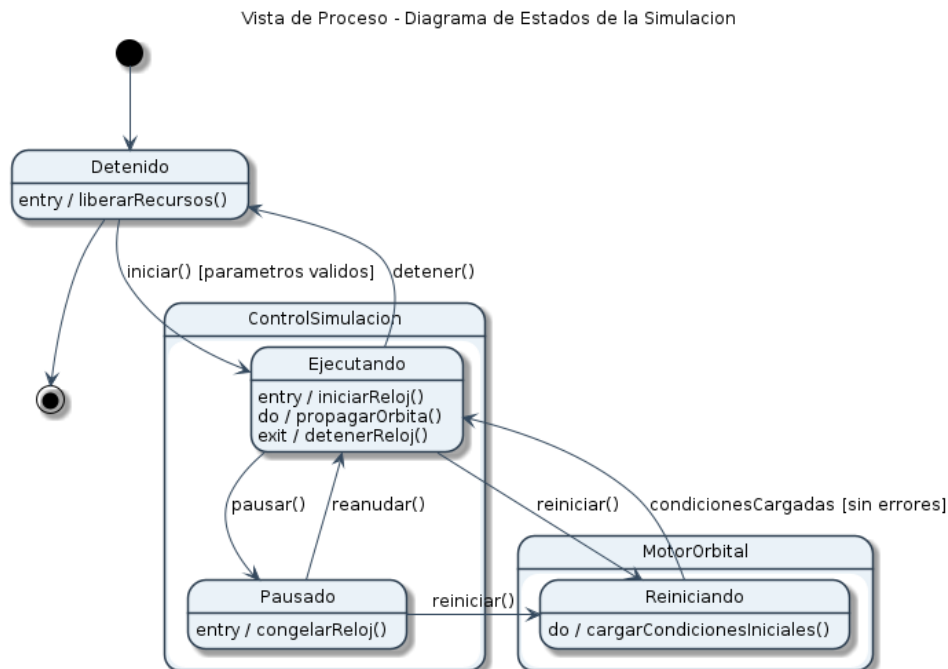


Figura 21. Diagrama de estados — ciclo de vida de la simulación.

7. Vista de desarrollo (implementación)

La vista de desarrollo representa la organización del código fuente y dependencias utilizadas. La estructura propuesta es:

```
src
├── app
│   └── Main.java
├── simulation
│   ├── SimulationController.java
│   └── MissionClock.java
├── orbital
│   ├── OrbitalEngine.java
│   └── OrbitalState.java
├── telemetry
│   └── TelemetryManager.java
├── alerts
│   └── AlertManager.java
└── ui
    ├── MainView.java
    └── ControlsPanel.java
```

Dependencias principales:

- JavaFX → Interfaz gráfica
- Orekit → Cálculo orbital
- JDK → Ejecución del sistema
- Git/GitHub → Control de versiones

Diagrama de Componentes

El diagrama siguiente organiza los componentes en tres capas lógicas (Presentación, Aplicación y Dominio Orbital) más las librerías externas, y hace explícitas las interfaces provistas y requeridas entre ellos: por ejemplo, `SimulationController` provee `IControlSimulacion` (consumida por `ControlsPanel` y `MainView`), y `OrbitalEngine` provee `IPropagacion` e `IDatosOrbitales`, a la vez que requiere `IPropagadorOrekit` de la librería Orekit. Las dependencias fluyen en una sola dirección (presentación → aplicación → dominio → librerías externas), sin ciclos.

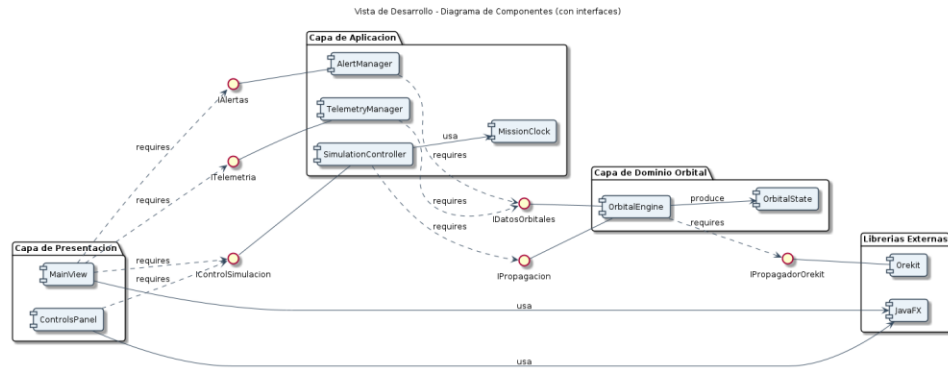


Figura 22. Diagrama de componentes — capas, interfaces provistas y requeridas.

8. Vista física

La vista física describe dónde se ejecutan los componentes del sistema. El despliegue inicial consiste en una aplicación de escritorio ejecutada en una única máquina.

Nodo Usuario

El equipo del operador se modela con el estereotipo «device», y dentro de él la JVM se modela como «execution environment» que aloja el artefacto ejecutable MisionOasisLunar.jar.

Nodo Aplicación

El artefacto MisionOasisLunar.jar manifiesta (empaqueta) los siguientes componentes de la vista de desarrollo:

- Interfaz JavaFX (MainView, ControlsPanel).
- Controlador de simulación (SimulationController).
- Motor orbital (OrbitalEngine).
- Sistema de telemetría (TelemetryManager).
- Sistema de alertas (AlertManager).

Nodo Bibliotecas

Las dependencias externas se modelan como artefactos dentro de un nodo «node» independiente (Sistema de Archivos Local): orekit-12.jar y javafx-sdk-21. La JVM se comunica con este nodo mediante el protocolo «classpath» — carga local de bibliotecas en tiempo de ejecución, sin red — representado como una asociación dirigida y etiquetada entre ambos nodos.

Diagrama de Despliegue

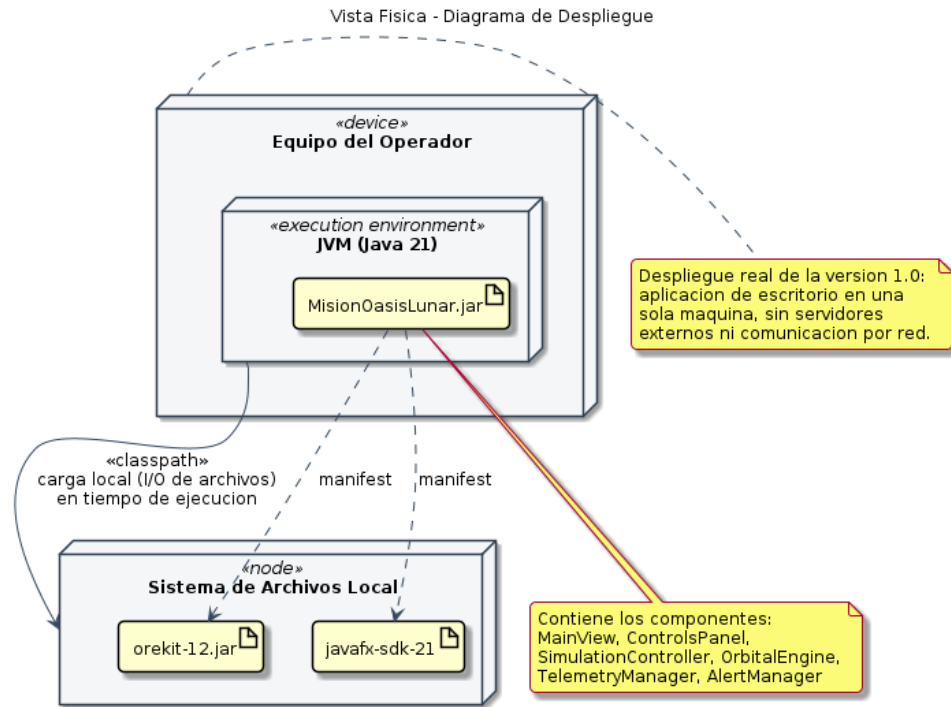


Figura 23. Diagrama de despliegue — Misión Oasis Lunar.

La primera versión no requiere servidores externos ni comunicación por red; el diagrama refleja el entorno de despliegue real proyectado para la versión 1.0.

9. Conclusión

La arquitectura propuesta para Misión Oasis Lunar permite desarrollar un simulador espacial organizado, extensible y basado en buenas prácticas de ingeniería de software.

El uso del modelo 4+1 permite analizar el sistema desde diferentes perspectivas, facilitando la comprensión de su estructura, comportamiento e implementación. Cada vista —lógica, de proceso, de desarrollo, física y de escenarios— aporta una perspectiva complementaria que, en conjunto, ofrece una visión completa y coherente de la arquitectura del sistema.

La separación entre interfaz, simulación orbital y procesamiento de datos permite mantener un diseño flexible que puede evolucionar hacia futuras versiones con funcionalidades más avanzadas.

Como mejoras futuras se recomienda incorporar:

- Visualización tridimensional.
- Simulación de múltiples vehículos espaciales.
- Integración con fuentes externas de datos orbitales.
- Modelos físicos más avanzados.
- Sistemas de análisis automático de misión.

La arquitectura definida proporciona una base sólida para continuar el desarrollo del proyecto y cumplir los objetivos establecidos para el simulador de misión lunar.